

SOFTWARE REQUIREMENTS SPECIFICATION

Globe Trotter – Personalized Multi-City Travel Planning Platform

[STUDENT NAME] – [ROLL NUMBER]

[STUDENT NAME] – [ROLL NUMBER]

[STUDENT NAME] – [ROLL NUMBER]

[STUDENT NAME] – [ROLL NUMBER]

1.0. Introduction

Globe Trotter is a full-stack, web-based personalized travel planning application. It allows users to create and manage multi-city trips, search for destinations and tourist attractions, build day-wise itineraries, receive transport recommendations, estimate trip expenses, and visualize their journey using maps and calendar/timeline views. Globe Trotter is a travel planning platform and not a booking platform; it does not provide flight, hotel, train, bus, or activity booking, payment processing, or ticket reservations, and all costs displayed are estimated values intended only for trip planning.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of the Globe Trotter travel planning platform. This document serves as a guideline for the design, development, testing, and deployment of the system. It also ensures a common understanding of the project among developers and faculty members for both the Agile Software Development and UI/UX Design laboratory requirements.

1.2 Scope of the Project

The proposed system provides a single, centralized platform for personalized, multi-city travel planning, replacing the fragmented process of using multiple applications and websites for destination discovery, itinerary preparation, transport comparison, budgeting, and route visualization.

The system includes:

- User registration and authentication
- Trip creation and management
- Dynamic destination search and selection
- Multi-city itinerary building
- Activity search and day-wise activity planning
- Transport recommendation between cities
- Budget estimation with category-wise breakdown
- Interactive map visualization
- Calendar and timeline itinerary views
- Trip sharing and copying

The project will be developed as a responsive web application and deployed on the cloud.

1.3 Glossary

Term	Definition
Trip	A user-created travel plan consisting of one or more cities, dates, activities, transport choices, and an estimated budget.
Itinerary	The complete day-wise plan of cities, activities, and transport that make up a trip.
API	Application Programming Interface – a web service architecture that enables communication between the frontend, backend, and external services.
External Location API	A third-party service (e.g., OpenStreetMap/Nominatim, Google Places) used to dynamically retrieve destination and attraction information.
JWT	JSON Web Token – a secure authentication mechanism used for verifying and maintaining user sessions.
PostgreSQL	Relational Database Management System used to store application-specific, user-selected trip data.
Itinerary Builder	The core module that allows users to add, remove, and reorder multiple cities within a trip.
Budget Estimation	The module that calculates an estimated total and category-wise cost breakdown for a trip.
Leaflet.js	A JavaScript library used with OpenStreetMap to render interactive maps and travel routes.
Shared Trip	A read-only, publicly accessible view of a completed itinerary generated for sharing purposes.

1.4 References

- IEEE Software Requirements Specification (SRS) Guidelines.
- OpenStreetMap Nominatim API Documentation.
- Google Places API Documentation.
- Leaflet.js Official Documentation.

2.0 Overall Description

Globe Trotter is a web-based platform designed to simplify and centralize personalized, multi-city travel planning. The system enables users to create trips, dynamically search and select destinations, arrange multiple cities in travel order, add and schedule activities, receive

transport recommendations, estimate the overall trip budget, visualize the route on a map, and share the completed itinerary with others.

2.1 Product Perspective

Globe Trotter is a standalone web application that serves as a centralized travel-planning platform for individual users. It integrates external location and places APIs for dynamic worldwide destination and attraction data, a relational database for user-specific application data, and an interactive mapping service for route visualization. It is designed with a modular architecture, allowing future integration with additional travel-related services.

The platform will interface with:

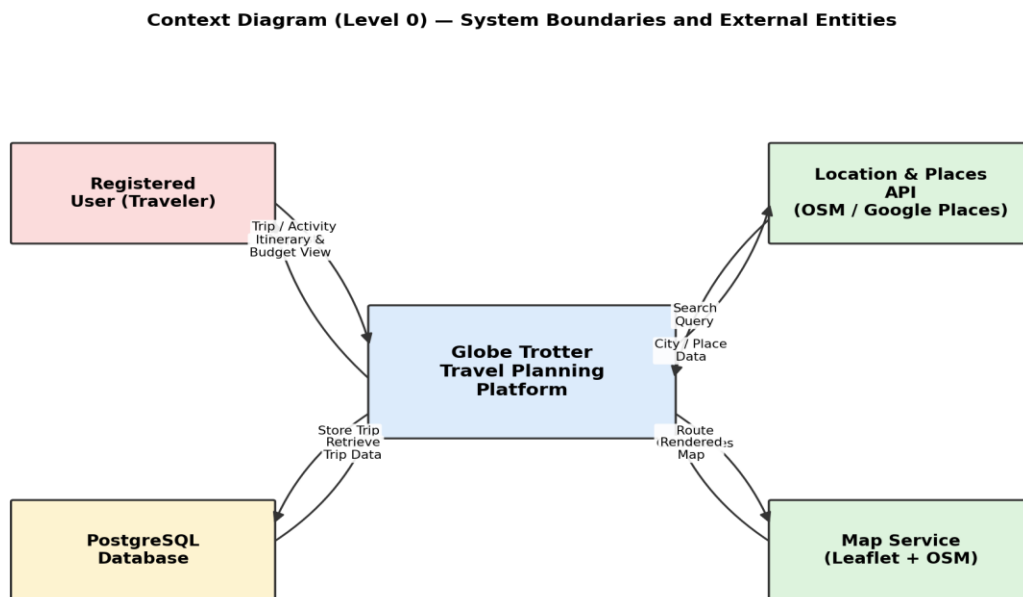


Figure 2.1 – Context Diagram: System Boundaries and External Entities

2.2 Product Functions

The proposed system provides the following major functionalities:

- User Registration and Authentication
- Trip Creation and Management
- Dynamic Destination Search and Selection
- Multi-City Itinerary Builder
- Activity Search and Day-wise Activity Planning

- Transport Recommendation Between Cities
- Budget Estimation with Category-wise Breakdown
- Interactive Map Visualization
- Calendar and Timeline Itinerary View
- Trip Sharing and Copy Trip
- Dashboard and User Profile Management

2.3 User Classes and Characteristics

The system supports a single primary user class, the Registered User (Traveler), since Globe Trotter is a personal travel-planning tool rather than a multi-role enterprise system.

Registered User (Traveler)

- Create and manage personal trips
- Search and select destinations and activities
- Build and reorder a multi-city itinerary
- Select transport options and view estimated budgets
- View itinerary on calendar, timeline, and map
- Share or copy trips, and manage their profile

2.4 Operating Environment

Software Environment

- React.js (Frontend)
- Node.js with Express.js (Backend)
- PostgreSQL (Database)
- Leaflet.js with OpenStreetMap (Map Visualization)
- External Location/Places API (e.g., OpenStreetMap Nominatim, Google Places)

Hardware Environment

- Desktop or Laptop
- Smartphone or Tablet (Responsive Web Application)
- Internet Connection

Browser Support

- Google Chrome
- Mozilla Firefox

- Microsoft Edge
- Safari

2.5 Design and Implementation Constraints

- The application will be developed as a web-based system.
- Destination and activity search functionality depends on the availability of external location APIs.
- Stable internet connectivity is required for cloud deployment and external API access.
- The application does not provide booking, payment, or ticket reservation functionality of any kind.
- The project will be completed within the Agile semester timeline.

2.6 User Documentation

The system will provide the following documentation:

- User Guide
- Installation Guide
- API Documentation
- Frequently Asked Questions (FAQ)

2.7 Assumptions and Dependencies

Assumptions:

- Users have basic knowledge of web applications.
- Users provide accurate trip dates and destination selections.
- Internet connectivity is available during system usage.

Dependencies:

- External location/places API for destination and attraction search.
- PostgreSQL database for application data storage.
- Leaflet.js and OpenStreetMap for map visualization.
- Cloud platforms for hosting and deployment.

3.0 System Features and Functional Requirements

This section describes the functional requirements of Globe Trotter. The requirements are organized based on the major system modules. Each requirement is assigned a unique identifier (FR-XX) to ensure traceability throughout the software development lifecycle.

3.1 User Authentication

ID	Requirement
FR-01	The system shall allow users to register using their email address and password.
FR-02	The system shall support secure user login and logout functionality.
FR-03	The system shall implement JWT-based authentication for all protected resources.
FR-04	The system shall securely hash user passwords using bcrypt before storage.
FR-05	The system shall authorize access so that users can view, modify, and delete only their own private trips.
FR-06	The system shall allow users to log out and invalidate their active session.

3.2 Trip Creation and Management

ID	Requirement
FR-07	The system shall allow users to create a trip with a name, start date, end date, description, and optional cover image.
FR-08	The system shall allow users to view a list of all their created trips.
FR-09	The system shall allow users to edit trip details after creation.
FR-10	The system shall allow users to delete a trip.
FR-11	The system shall allow users to open an existing trip and continue planning it.

3.3 Destination Search and Selection

ID	Requirement
FR-12	The system shall allow users to dynamically search for destinations using an external location API (e.g., OpenStreetMap/Nominatim or Google Places).
FR-13	The system shall display destination search results including city name, country, coordinates, description, and image where available.
FR-14	The system shall allow users to select a destination and add it to the current trip.
FR-15	The system shall store only the destinations selected by the user, rather than maintaining a global destination database.

3.4 Multi-City Itinerary Builder

ID	Requirement
FR-16	The system shall allow users to add multiple destinations/cities to a trip.

ID	Requirement
FR-17	The system shall allow users to remove a destination from a trip.
FR-18	The system shall allow users to reorder destinations within a trip.
FR-19	The system shall allow users to set arrival and departure dates for each destination.
FR-20	The system shall display destinations in travel order within the itinerary.
FR-21	The system shall provide both a list view and a timeline/day-wise view of the itinerary.

3.5 Activity Search and Day-wise Planning

ID	Requirement
FR-22	The system shall allow users to search for tourist attractions and activities associated with a selected destination, preferably through an external API.
FR-23	The system shall display activity information including name, description, location, coordinates, image, and estimated cost where available.
FR-24	The system shall allow users to select activities and add them to the itinerary.
FR-25	The system shall allow users to assign selected activities to specific trip dates.
FR-26	The system shall allow users to remove activities from the itinerary.

3.6 Transport Recommendation

ID	Requirement
FR-27	The system shall display estimated transport options (e.g., bus, train, flight) between consecutive cities in the itinerary, including estimated cost and duration.
FR-28	The system shall allow users to select one transport option between each pair of consecutive cities.
FR-29	The system shall associate the selected transport option with the corresponding city-to-city journey and include it in the itinerary.
FR-30	The system shall treat all transport information as an estimated planning value and shall not book tickets, reserve seats, process payments, or guarantee availability or pricing.

3.7 Budget Estimation

ID	Requirement
FR-31	The system shall calculate an estimated trip budget across the categories of accommodation, food, local transport, intercity transport, activities, and miscellaneous expenses.
FR-32	The system shall display the total estimated cost of the trip.
FR-33	The system shall display the average estimated cost per day.
FR-34	The system shall display a category-wise breakdown of the estimated budget.
FR-35	The system shall visually represent the budget breakdown using charts.

3.8 Map Visualization

ID	Requirement
FR-36	The system shall display an interactive map showing markers for selected cities and activities.
FR-37	The system shall display the travel route connecting selected cities in itinerary order on the map.
FR-38	The system shall use the map only for visualization purposes and shall not provide turn-by-turn navigation.

3.9 Calendar and Timeline View

ID	Requirement
FR-39	The system shall provide a calendar view showing activities according to their scheduled dates.
FR-40	The system shall provide a timeline view showing a chronological representation of the complete journey.
FR-41	The system shall display intercity travel segments between destinations within the calendar/timeline view.

3.10 Trip Sharing

ID	Requirement
FR-42	The system shall allow users to generate a public sharing link for a completed itinerary.
FR-43	The system shall provide a read-only itinerary view for users who access a shared

ID	Requirement
	link.
FR-44	The system shall prevent external viewers of a shared itinerary from modifying the original owner's trip.
FR-45	The system shall allow a viewer to use a “Copy Trip” function to create their own editable copy of a shared itinerary.

3.11 Dashboard

ID	Requirement
FR-46	The system shall display a dashboard with a welcome message, recent trips, and upcoming trips.
FR-47	The system shall display popular destination suggestions on the dashboard.
FR-48	The system shall provide a “Plan New Trip” action on the dashboard.

3.12 User Profile

ID	Requirement
FR-49	The system shall allow users to view their profile information.
FR-50	The system shall allow users to edit their profile information and change their password.
FR-51	The system shall allow users to upload or update a profile picture.
FR-52	The system shall allow users to view a list of their saved trips from their profile.

4.0. External Interface Requirements

This chapter describes the external interfaces of Globe Trotter. It specifies how users interact with the system and how the system communicates with external software services and hardware components.

4.1 User Interfaces

The platform provides a responsive, user-friendly web interface designed to ensure easy navigation, accessibility, and efficient interaction across desktops, laptops, tablets, and mobile devices.

The system includes the following user interfaces:

- Login and Registration
- Dashboard
- Create Trip
- Destination Search
- Itinerary Builder
- Activity Search
- Budget View
- Calendar / Timeline View
- Map View
- Trip Details
- Shared Trip (Read-only)
- User Profile

4.2 Hardware Interfaces

The proposed system does not require any specialized hardware. It operates using standard computing devices with internet connectivity. Minimum hardware requirements include a desktop computer, laptop, smartphone, or tablet with a keyboard/touch input and an internet connection.

Recommended system configuration:

Component	Specification
Processor	Intel Core i3 / AMD Ryzen 3 or higher
RAM	4 GB minimum (8 GB recommended)
Storage	1 GB available space
Internet	Broadband Connection

4.3 Software Interfaces

The system communicates with several software components and third-party services to provide destination discovery, itinerary planning, and map visualization functionality.

Software	Purpose
React.js	Frontend development

Software	Purpose
Node.js / Express.js	Backend REST APIs
PostgreSQL	Database management
Leaflet.js	Interactive map rendering
OpenStreetMap	Base map tiles and geocoding
Chart.js	Budget visualization charts
JWT	User authentication

4.4 Communication Interfaces

The system communicates over the Internet using secure communication protocols.

Communication specifications include:

- HTTP/HTTPS protocol for client-server communication.
- RESTful APIs for frontend-backend communication.
- JSON as the standard data exchange format.
- Secure authentication using JWT tokens.
- Cloud-based communication with external location/places APIs for destination and activity data.
- Communication with map tile services for map rendering.

5.0. Non-Functional Requirements

This chapter describes the quality attributes and operational characteristics of Globe Trotter. These requirements define how the system should perform and ensure reliability, security, usability, maintainability, and scalability.

5.1 Performance Requirements

5.1.1 Response Time

ID	Requirement
NFR – 01.	The system shall respond to user requests within 3 seconds under normal operating conditions.
NFR – 02.	Destination and activity search results shall be displayed within 5 seconds under normal network conditions.
NFR – 03.	Budget calculations shall be updated within 2 seconds after a transport or activity selection changes.

ID	Requirement
NFR – 04.	Map visualization shall render within 3 seconds after itinerary changes.

5.1.2 Scalability

ID	Requirement
NFR – 05.	The system shall support multiple users accessing the application simultaneously.
NFR – 06.	The architecture shall support future expansion with additional travel-related features without significant modification.
NFR – 07.	The database shall support increasing volumes of trips, destinations, and itinerary records.

5.1.3 Availability

ID	Requirement
NFR – 08.	The system shall be available 24 hours a day, 7 days a week, except during scheduled maintenance.
NFR – 09.	Scheduled maintenance activities shall be communicated to users in advance.

5.2 Security Requirements

5.2.1 Authentication and Authorization

ID	Requirement
NFR – 10.	The system shall implement secure user authentication using JWT-based authentication.
NFR – 11.	The system shall restrict access so that users may only view, modify, or delete their own private trips.
NFR – 12.	Passwords shall be securely stored using bcrypt hashing.

5.2.2 Data Protection

ID	Requirement
NFR – 13.	All communication between the client and server shall use HTTPS.
NFR – 14.	User and trip information shall be securely stored in the database.
NFR – 15.	Only authorized users shall access their own trip and profile data.

5.3 Reliability

ID	Requirement
NFR – 16.	The system shall provide consistent and accurate itinerary, transport, and budget calculations.
NFR – 17.	The application shall recover gracefully from unexpected failures without data loss.
NFR – 18.	Error messages shall be displayed clearly without exposing sensitive system information.
NFR – 19.	The application shall handle external API failures gracefully and inform the user when destination or activity data cannot be retrieved.

5.4 Usability

ID	Requirement
NFR – 20.	The system shall provide a simple and intuitive user interface.
NFR – 21.	The application shall support responsive design for desktop, tablet, and mobile screens.
NFR – 22.	Navigation shall be consistent across all modules.
NFR – 23.	Users shall be able to create a trip and add a destination with a minimal number of steps.

5.5 Maintainability

ID	Requirement
NFR – 24.	The application shall follow a modular frontend and backend architecture.
NFR – 25.	Source code shall be properly documented and organized.
NFR – 26.	New features shall be added without affecting existing functionality.
NFR – 27.	APIs shall be designed following RESTful standards.

5.6 Compatibility

ID	Requirement
NFR – 28.	The system shall support modern web browsers including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
NFR – 29.	The application shall be compatible with Windows, Linux, Android, iOS, and macOS through a web browser.

5.7 Backup and Recovery

ID	Requirement
NFR – 30.	The database shall be backed up periodically.
NFR – 31.	The system shall be able to restore trip and user data in case of unexpected failure.

5.8 Operational Requirements

ID	Requirement
NFR – 32.	The application shall be deployed on a cloud platform.
NFR – 33.	The system shall support continuous monitoring and logging of API usage and errors.
NFR – 34.	The application shall be accessible through a stable internet connection.
NFR – 35.	Future updates shall be deployed with minimal service interruption.

5.9 Legal and Privacy Requirements

ID	Requirement
NFR – 36.	User information shall be collected only for trip-planning purposes.
NFR – 37.	Personal information shall not be shared with unauthorized users.
NFR – 38.	A shared trip link shall expose only itinerary information the owner intends to share, and not the owner's account or profile details.
NFR – 39.	The system shall maintain the confidentiality of user account data.

5.10 External API and Data Accuracy Requirements

ID	Requirement
NFR – 40.	Destination, activity, and coordinate information shall be retrieved dynamically from external location/places APIs rather than a locally maintained global database.
NFR – 41.	Transport and budget estimates shall be clearly presented as planning estimates and not as guaranteed or real-time prices.
NFR – 42.	The system shall degrade gracefully and continue to function using locally stored trip data if an external API is temporarily unavailable.

6.0. Other Requirements

6.1 Deployment Requirements

Globe Trotter shall be deployed as a cloud-based web application to ensure accessibility, scalability, and reliability. The frontend application will be deployed using a static hosting platform (e.g., Vercel or Netlify), while the backend services will be hosted on a cloud application platform (e.g., Render). The application database will be managed using a hosted PostgreSQL service. The deployment architecture shall support secure communication between all components using HTTPS.

6.2 Data Management Requirements

The system shall securely manage all user and application data throughout its lifecycle. User profiles, trips, selected cities, activities, selected transport options, and expense records shall be stored in a relational database. Only user-selected destination and activity data shall be persisted; the system shall not attempt to mirror external APIs' full datasets locally. Regular database backups shall be maintained to ensure data availability and recovery in case of failures.

6.3 External API and Estimation Requirements

The system shall depend on external location and places APIs for destination search, attraction/activity data, and geographic coordinates. Hotel, food, and local transportation costs shall be estimated using predefined cost tiers, travel-cost indices, or configurable estimation rules, since exact real-time pricing cannot be maintained for destinations worldwide. Intercity transport costs shall be based on the user's selected estimated transport option. All resulting budgets are planning estimates only and are not booking quotes.

6.4 Training and User Support

The system shall provide sufficient documentation and guidance for users, including a user guide, FAQs, and help pages to understand system functionality. The application shall include informative error messages and user-friendly interfaces to minimize the need for external technical support.

6.5 Future Enhancements

Globe Trotter is designed with a modular architecture to support future enhancements without affecting existing functionality. Future improvements may include:

- Optional integration with third-party booking and payment providers.
- Collaborative multi-user trip editing.

- Offline itinerary export (PDF/print-friendly view).
- Email or push notifications for trip reminders.
- Multi-language and multi-currency support.
- Native mobile application support.
- AI-assisted itinerary and activity suggestions.

6.6 Appendix A: Analysis Models

This appendix includes the following analysis models:

1) Context Diagram

- System boundaries and external entities (Level 0)

Context Diagram (Level 0) — System Boundaries and External Entities

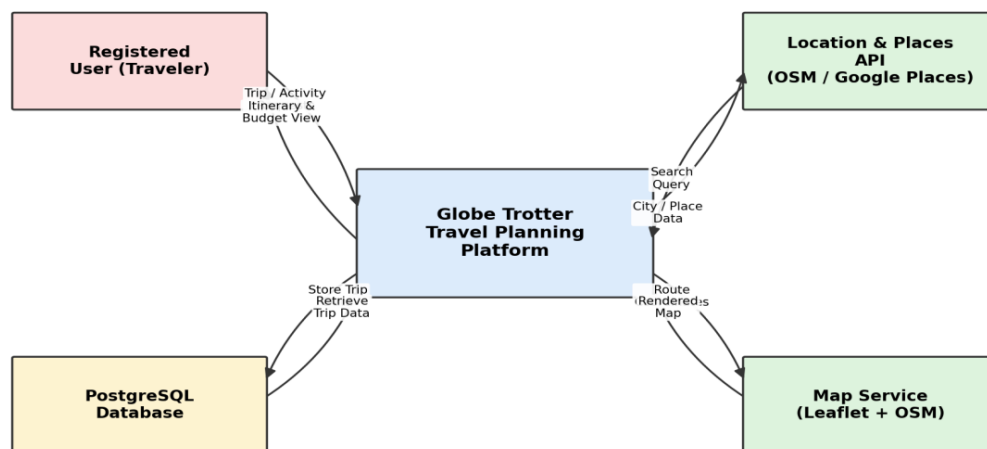


Figure A.1 – Context Diagram (Level 0)

2) Data Flow Diagram

- Data flows between the system and external entities (Level 1)

Data Flow Diagram (Level 1) – Flows Between Modules and External Entities

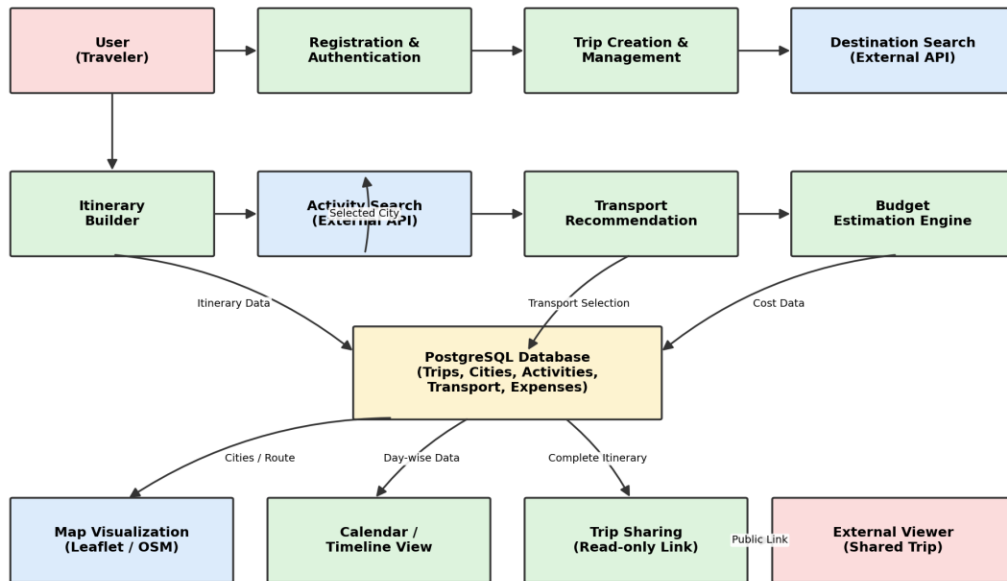


Figure A.2 – Data Flow Diagram (Level 1)

3) Process Flow Diagram

Process Flow Diagram — Trip Planning Workflow

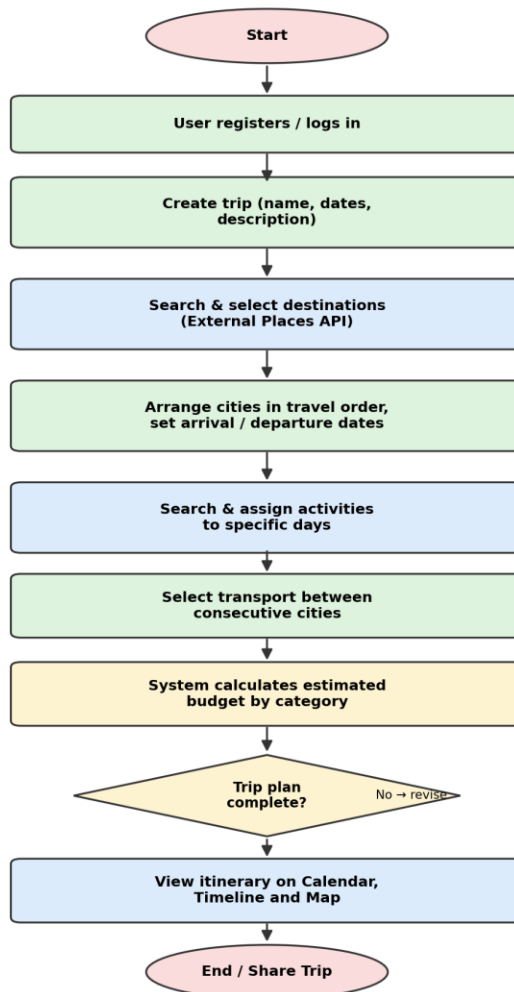


Figure A.3 – Process Flow Diagram: Trip Planning Workflow

4) Entity Relationship Diagram

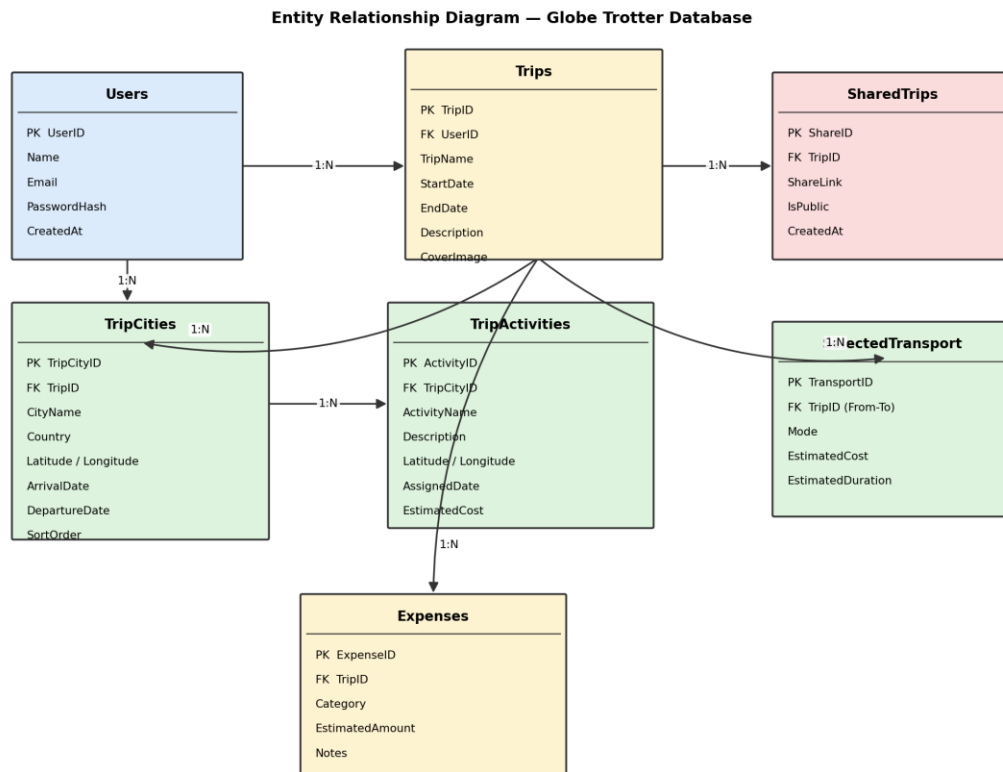


Figure A.4 – Entity Relationship Diagram – Globe Trotter Database

5) Use Case Diagram

Key system use case: Registered User (Traveler)

Use Case Diagram — Registered User (Traveler)

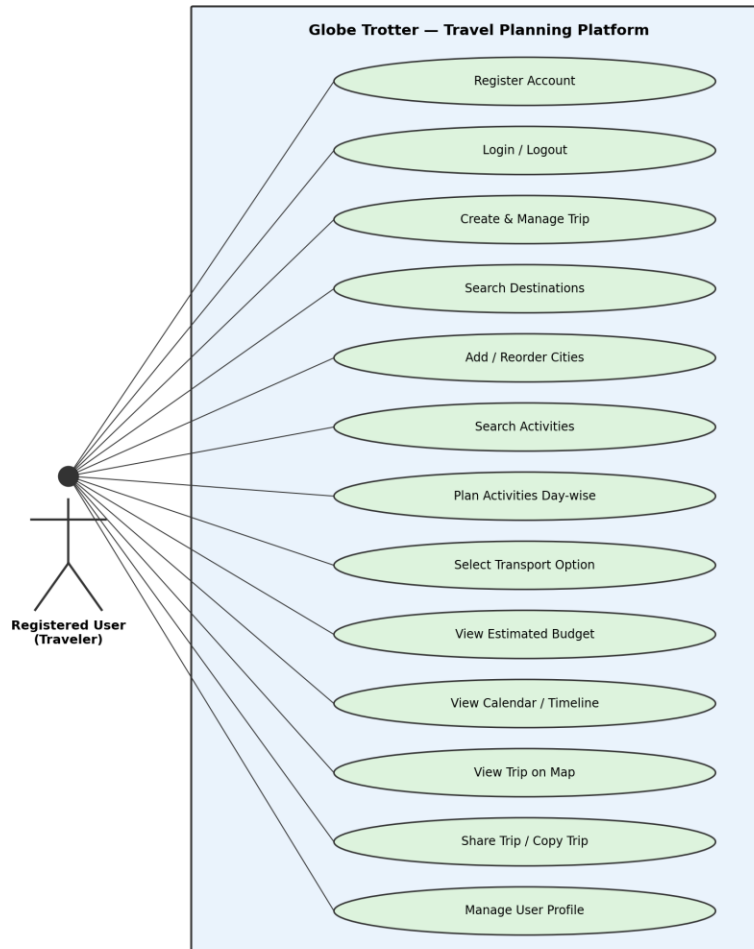


Figure A.5 – Use Case Diagram: Registered User (Traveler)

6.6 Appendix B: Issues List

Appendix B lists the current limitations, pending features, and future improvements identified during the development of Globe Trotter. These items are planned for future iterations and do not affect the core functionality of the current release.

ISSUE ID	DESCRIPTION	STATUS
IS – 01.	Integration with third-party booking and payment providers	Future Work
IS – 02.	Collaborative multi-user trip editing	Future Work
IS – 03.	Offline itinerary export (PDF/print-friendly view)	Planned
IS – 04.	Email/push notification service for trip reminders	Planned

ISSUE ID	DESCRIPTION	STATUS
IS – 05.	Multi-language and multi-currency support	Future Work
IS – 06.	Native mobile application development	Future Work